Saga

Season: 4 · Enterprise · Category: Enterprise · Level: Intermediate

Intent

- Coordinate distributed steps with compensations.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Travel booking: flight + hotel, cancel if payment fails.

When to Use

- Microservices / modular monoliths.
- The design force named in Intent shows up more than once.

When Not to Use

- Greenfield CRUD without distribution.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.enterprise.saga;

import java.util.ArrayList;
import java.util.List;
import java.util.function.Supplier;

/** Minimal choreography-style saga with compensations. */
public final class OrderSaga {
    private final List<Runnable> compensations = new ArrayList<>();
    private final List<String> log = new ArrayList<>();

    public void step(String name, Runnable action, Runnable compensate) {
        action.run();
        log.add("ok:" + name);
        compensations.addFirst(compensate);
    }

    public void run(Supplier<Boolean> paymentOk) {
        try {
            step("reserve", () -> log.add("reserved"), () -> log.add("release"));
            step("charge", () -> {
                if (!paymentOk.get()) throw new IllegalStateException("pay");
                log.add("charged");
            }, () -> log.add("refund"));
            step("ship", () -> log.add("shipped"), () -> log.add("recall"));
        } catch (RuntimeException e) {
            for (var c : compensations) c.run();
            log.add("aborted");
        }
```

Classic Implementation (2)

```
    }

    List<String> log() { return List.copyOf(log); }
}
```

Modern Java 25 / Functional

- Sealed events + pattern matching for projections.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: enterprise/saga/
```

Pitfalls

- Keep demos minimal — production needs idempotency and observability.
- Saga compensations must be careful.

Interview Q&A

- Q: Production concern for Saga? A: Idempotency, observability, and failure modes.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Coordinate distributed steps with compensations.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Travel booking: flight + hotel, cancel if payment fails. [0:40] Intent — State the design force in o...